Reconnaissance de caractères manuscrits tifinagh par un LeNet-5 adapté aux images RGB

Abdelmajid Ouchni
Master Intelligence Artificielle & Applications
Faculté des Sciences Appliquées d'Ouarzazate
Encadrant : Dr. Mohamed BENADDY

***Résumé*—La reconnaissance de caractères manuscrits tifinagh constitue un problème de classification visuelle dans lequel des formes proches peuvent être difficiles à distinguer. Cet article étudie une adaptation RGB de LeNet-5 sur la base AMHCD, composée dans l'expérience de 25 740 images réparties de façon équilibrée entre 33 classes. Les images sont redimensionnées en 32×32 pixels, normalisées dans $[0, 1]$, puis séparées en ensembles d'entraînement, de validation et de test selon un protocole stratifié de 60 %, 20 % et 20 %. Deux configurations d'optimisation sont comparées avec la même architecture, les mêmes partitions et des lots de 64 images : SGD avec momentum et un taux d'apprentissage initial de 10^{-2}, et Adam avec un taux de 10^{-3}. Sur les 5 148 images de test, SGD obtient une exactitude de 97,94 % et un F1 macro de 0,9794, contre 96,89 % et 0,9689 pour Adam. La perte de test est respectivement de 0,0773 et 0,1166. Les courbes d'apprentissage, la matrice de confusion, des prédictions qualitatives et des cartes de caractéristiques complètent l'analyse. Les résultats favorisent SGD pour les deux configurations testées, sans constituer une recherche exhaustive des hyperparamètres ni une preuve de généralisation indépendante par scripteur.**

***Index Terms*—Tifinagh, AMHCD, reconnaissance de caractères manuscrits, réseau de neurones convolutif, LeNet-5, SGD, Adam, classification d'images.**

I. Introduction

La reconnaissance optique de caractères manuscrits (HCR) vise à associer une image d'écriture à une classe symbolique. Pour le tifinagh, les variations individuelles de tracé et la proximité visuelle entre certains caractères rendent utile l'apprentissage de caractéristiques hiérarchiques à partir des pixels. Les réseaux de neurones convolutifs (CNN) apprennent de telles représentations locales et ont joué un rôle majeur dans la reconnaissance de documents, notamment avec l'architecture LeNet-5 proposée par LeCun et ses collègues [2].

La base Amazigh Handwritten Character Database (AMHCD) a été présentée comme une ressource de recherche pour la reconnaissance de caractères amazighs manuscrits [1]. Dans cette étude, nous utilisons une version organisée de 25 740 images réparties sur 33 classes. Le protocole étudié conserve les trois canaux RGB et adapte les dimensions de l'entrée aux contraintes du TP. La contribution principale est double : décrire précisément l'architecture et son protocole expérimental, puis comparer quantitativement SGD et Adam à partir des résultats enregistrés dans le notebook associé.

Les questions étudiées sont les suivantes : (1) quel niveau de performance la version RGB de LeNet-5 atteint-elle sur la partition de test retenue ? et (2) quelle différence observe-t-on entre les configurations SGD et Adam sélectionnées, en matière d'exactitude, de perte, de F1 macro et de dynamique de validation ?

Table I
Détail des couches et des paramètres entraînables.

Couche	Sortie	Paramètres
Entrée	$32 \times 32 \times 3$	0
C1 : Conv. 5×5, 6	$28 \times 28 \times 6$	456
S2 : AveragePool 2×2	$14 \times 14 \times 6$	0
C3 : Conv. 5×5, 16	$10 \times 10 \times 16$	2 416
S4 : AveragePool 2×2	$5 \times 5 \times 16$	0
Flatten	400	0
C5 : Dense, 120	120	48 120
F6 : Dense, 84	84	10 164
Sortie : Dense, 33	33	2 805
Total		**63 961**

II. Matériels et méthodes

A. Données et prétraitement

Le corpus exploité contient 25 740 images, soit 780 images par classe dans les fichiers analysés. Le nombre d'échantillons est équilibré entre les 33 classes. Les images sont lues en couleur, converties de BGR vers RGB, redimensionnées en 32×32 pixels, puis les valeurs des pixels sont ramenées à l'intervalle $[0, 1]$ par division par 255. Chaque image est ainsi représentée par un tenseur de dimension $32 \times 32 \times 3$.

La séparation des données est stratifiée et utilise une graine aléatoire fixée à 42. Elle comprend 15 444 images d'entraînement (60 %), 5 148 de validation (20 %) et 5 148 de test (20 %). Chaque classe compte donc 468 images d'entraînement, 156 de validation et 156 de test. L'ensemble de test est utilisé uniquement pour l'évaluation finale rapportée ; la validation permet de suivre l'apprentissage.

B. Architecture du réseau

Le modèle est inspiré de LeNet-5 et adapté à une entrée RGB et à 33 classes. Il comprend deux couches convolutives avec activation tangente hyperbolique, deux sous-échantillonnages par moyenne, puis trois couches denses. Les convolutions utilisent un remplissage `valid`, c'est-à-dire sans ajout de pixels autour de l'image. La dernière couche produit 33 logits ; la fonction de perte applique la normalisation softmax de manière intégrée et numériquement stable.

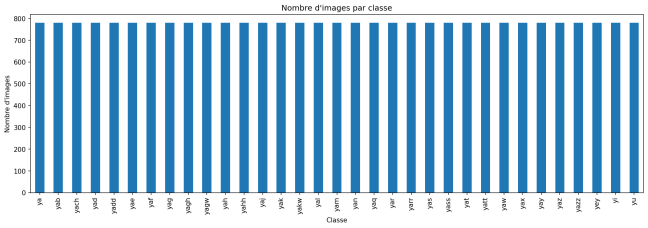


FIGURE 1. Distribution observée des images par classe. Les classes sont équilibrées, avec 780 images par classe dans le corpus utilisé.



FIGURE 2. Exemples d'images RGB et de leurs étiquettes dans le corpus.

Les dimensions spatiales sont calculées avec une convolution sans padding :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K}{S} \right\rfloor + 1, \tag{1}$$

où H_{in} est la dimension d'entrée, K la taille du noyau et S le pas. Pour les couches de pooling, $K = 2$ et $S = 2$. Le nombre de paramètres des couches denses inclut les biais.

C. Fonction de perte et métriques

Pour une image i, le modèle produit les logits $z_{i,c}$ pour chacune des $C = 33$ classes. Les probabilités sont obtenues par softmax :

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{j=1}^{C} \exp(z_{i,j})}. \tag{2}$$

La perte d'entropie croisée sur N exemples est

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i}, \tag{3}$$

où y_i désigne la classe réelle. L'exactitude est le rapport entre le nombre de prédictions correctes et le nombre d'exemples :

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(\hat{y}_i = y_i). \tag{4}$$

La précision, le rappel et la F1-score sont également calculés en moyenne macro, afin de donner le même poids à chacune des 33 classes.

D. Comparaison des optimiseurs

Les deux modèles comparés sont initialisés avec la même graine aléatoire et entraînés sur les mêmes partitions. La taille de lot est de 64 images et le nombre maximal d'époques est de 30. Les deux entraînements utilisent les callbacks configurés dans le notebook : arrêt anticipé sur la perte de validation et réduction du taux d'apprentissage en cas de plateau. Les paramètres initiaux consignés sont présentés au Tableau II.

SGD met à jour les paramètres à l'aide du gradient stochastique et d'un terme de momentum :

$$v_t = \mu v_{t-1} - \eta g_t, \qquad \theta_t = \theta_{t-1} + v_t, \tag{5}$$

où g_t est le gradient courant, η le taux d'apprentissage et $\mu = 0{,}9$ le momentum. Adam utilise des estimations des premier et second moments du gradient et leur correction de biais [3] ; son taux d'apprentissage initial est ici fixé à 10^{-3}. SGD est évalué avec un taux initial de 10^{-2}. Ces valeurs sont les configurations testées, pas les résultats d'une recherche exhaustive d'hyperparamètres.

III. RÉSULTATS

A. Résultats quantitatifs

Le Tableau II rapporte les valeurs issues du fichier CSV de comparaison. SGD atteint 97,94 % d'exactitude sur le test, contre 96,89 % pour Adam, soit un écart de 1,05 point. Cela correspond à 5 042 classifications correctes pour SGD et 4 988 pour Adam parmi les 5 148 exemples de test. La perte de test obtenue avec SGD (0,0773) est inférieure à celle d'Adam (0,1166), soit une réduction d'environ 33,7 % relativement à la perte d'Adam. Le F1 macro est de 0,9794 pour SGD et de 0,9689 pour Adam.

À la dernière époque, SGD présente aussi de meilleurs résultats sur la validation : exactitude de 97,55 % et perte de 0,0881, contre 96,66 % et 0,1337 pour Adam. Les temps mesurés (108,70 s et 108,65 s) sont presque identiques dans cette exécution ; l'écart de 0,05 s ne doit pas être interprété comme une différence pratique de vitesse sans répétitions sur un environnement documenté.

B. Dynamique d'apprentissage et généralisation

La Figure 6 présente les courbes du modèle de référence Adam : la perte diminue et l'exactitude augmente au fil des époques. À la fin de l'entraînement, l'exactitude d'entraînement approche 100 %, alors que la validation se situe autour de 96,7 %. Cet écart est compatible avec un surapprentissage.

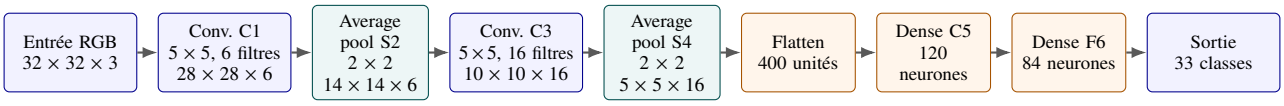


FIGURE 3. Schéma de l'architecture LeNet-5 adaptée aux images RGB. Le nombre total de paramètres entraînables est de 63 961.

TABLE II
COMPARAISON QUANTITATIVE DES CONFIGURATIONS SGD ET ADAM SUR L'ENSEMBLE DE TEST. LES TEMPS CORRESPONDENT AUX MESURES SAUVEGARDÉES DANS L'EXPÉRIENCE ET DÉPENDENT DE L'ENVIRONNEMENT D'EXÉCUTION.

Mesure	SGD	Adam	Écart (SGD − Adam)
Taux d'apprentissage initial	0,010	0,001	–
Momentum	0,9	–	–
Époques exécutées	30	30	0
Temps d'entraînement (s)	108,70	108,65	+0,05
Perte de test	**0,0773**	0,1166	−0,0393
Exactitude de test	**97,94 %**	96,89 %	+1,05 point
Précision macro	**0,9796**	0,9692	+0,0104
Rappel macro	**0,9794**	0,9689	+0,0105
F1-score macro	**0,9794**	0,9689	+0,0105
Prédictions correctes / 5 148	**5 042**	4 988	+54

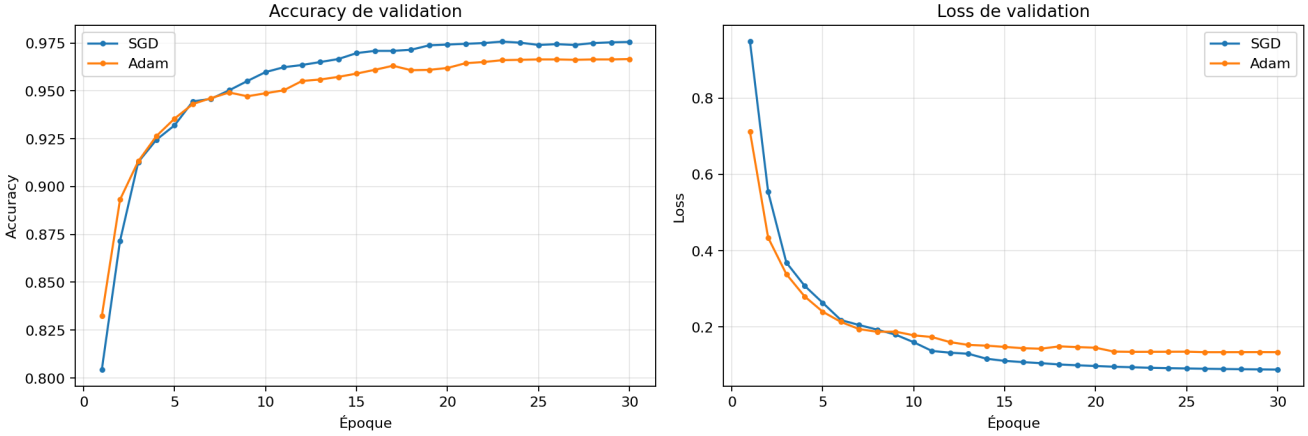


FIGURE 4. Évolution de l'exactitude et de la perte de validation pour les deux configurations. À la dernière époque, l'exactitude de validation enregistrée est de 97,55 % pour SGD et 96,66 % pour Adam ; les pertes correspondantes sont 0,0881 et 0,1337.

La comparaison directe de la Figure 4 suggère que, dans les configurations testées, SGD conserve une perte de validation plus faible et une exactitude de validation légèrement supérieure en fin d'entraînement.

C. Analyse par classe et matrice de confusion

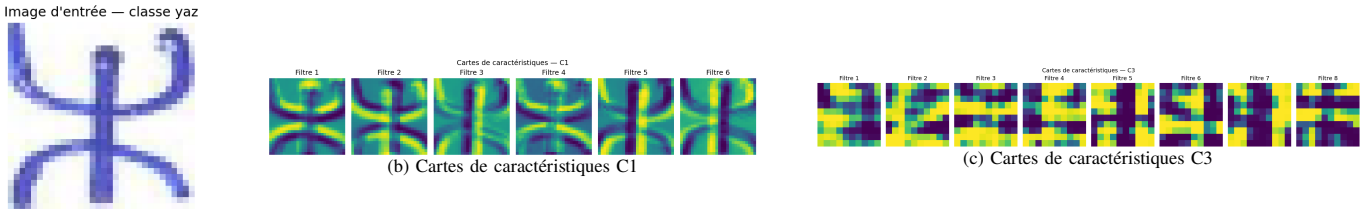
La matrice de confusion disponible concerne le modèle Adam de référence, dont l'exactitude est de 96,89 %. Sa diagonale dominante indique que la majorité des exemples sont correctement classés. Le rapport de classification correspondant donne un F1-score de 0,90 pour la classe `yaz` et de 0,92 pour `yazz`, parmi les valeurs les plus faibles. Les résultats montrent ainsi une performance élevée globale, mais non uniforme selon les classes. Aucune matrice de confusion distincte pour SGD n'a été fournie dans les résultats exploités ; la matrice ne doit donc pas être présentée comme une comparaison visuelle des deux optimiseurs.

D. Analyse qualitative des représentations

La Figure 8 donne des exemples de prédictions du modèle sur des images de test. Ces exemples illustrent le comportement du classifieur, mais ne remplacent pas les mesures quantitatives. Les cartes de caractéristiques de C1 et C3 (Figure 5) montrent des activations intermédiaires de plus en plus structurées. L'image d'entrée associée est rappelée dans le premier panneau. Ces visualisations permettent d'inspecter le traitement interne, sans suffire à elles seules pour expliquer causalement la décision de classification.

IV. DISCUSSION

Les résultats enregistrés indiquent que l'architecture LeNet-5 adaptée aux images RGB apprend des représentations utiles pour distinguer les 33 classes considérées. Pour les deux configurations testées, SGD surpasse Adam sur l'exactitude, la perte et le F1 macro. La différence d'exactitude, d'environ 1,05 point,



(a) Image d'entrée

(b) Cartes de caractéristiques C1

(c) Cartes de caractéristiques C3

FIGURE 5. Visualisation qualitative d'une image de test et des cartes de caractéristiques issues des couches convolutives C1 et C3.

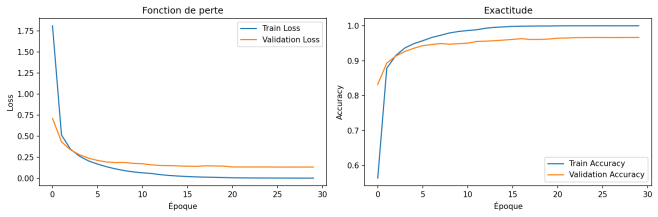


FIGURE 6. Courbes de perte et d'exactitude du modèle de référence entraîné avec Adam.

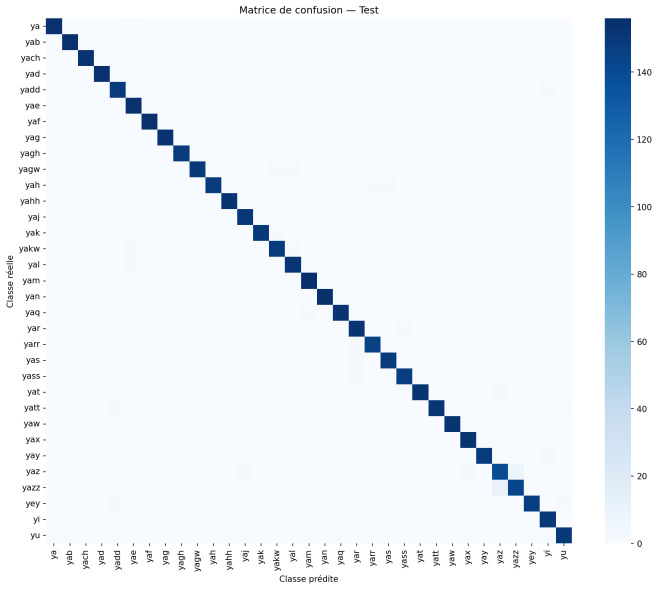


FIGURE 7. Matrice de confusion sur les 5 148 images de test pour le modèle Adam de référence.

correspond à 54 prédictions correctes supplémentaires sur le même ensemble de test. La proximité des temps d'entraînement suggère que le gain observé n'a pas été accompagné d'une hausse notable du temps mesuré dans cette exécution.

Cette conclusion doit toutefois être circonscrite. Premièrement, un seul taux d'apprentissage initial par optimiseur a été testé : le résultat compare deux configurations choisies, et non les performances intrinsèques de SGD et d'Adam après optimisation de leurs hyperparamètres. Deuxièmement, les fichiers fournis ne décrivent pas complètement le matériel d'exécution ; les temps ne sont donc pas transférables à d'autres machines. Troisièmement, la séparation train/validation/test est stratifiée au niveau des images, mais les fichiers ne permettent pas de confirmer une séparation indépendante par scripteur.



FIGURE 8. Exemples de prédictions illustratives sur l'ensemble de test.

Si l'identité d'un auteur se retrouve dans plusieurs partitions, les performances peuvent surestimer la capacité à reconnaître l'écriture d'une personne jamais vue.

Enfin, le rapport détaillé par classe et la matrice de confusion disponibles sont ceux du modèle Adam de référence. Une comparaison plus complète demanderait de sauvegarder séparément les prédictions de chaque optimiseur, de produire une matrice de confusion pour chacun, de répéter les entraînements avec plusieurs graines et d'effectuer une recherche d'hyperparamètres sous protocole identique. Une séparation par scripteur, si les métadonnées nécessaires sont disponibles, renforcerait l'évaluation de généralisation.

V. CONCLUSION

Cet article a décrit une adaptation RGB de LeNet-5 pour la reconnaissance de 33 classes de caractères manuscrits tifinagh issus de l'expérience AMHCD. Le corpus comprend 25 740 images et la partition de test 5 148 images. Avec les configurations enregistrées, SGD (taux initial 0,01, momentum 0,9) atteint 97,94 % d'exactitude et un F1 macro de 0,9794, tandis qu'Adam (taux initial 0,001) atteint 96,89 % et 0,9689. SGD donne également une perte de test inférieure. Ces résultats désignent SGD comme le meilleur des deux réglages évalués, sans permettre de conclure à une supériorité générale de cet optimiseur. Les travaux futurs devraient inclure plusieurs répétitions, une recherche des hyperparamètres, des matrices

de confusion pour chaque configuration et une séparation indépendante par scripteur.

Code et reproductibilité

Le notebook d'entraînement, la comparaison SGD/Adam et les fichiers du projet sont disponibles dans le dépôt GitHub de l'auteur :
https://github.com/ABdelmajid05/LeNet-5-for-AMHCD-Tifinagh-Character-Recognition.

Pour reproduire l'expérience, il faut disposer de la base AMHCD, adapter les chemins d'accès dans le notebook et exécuter les cellules dans l'ordre. Les figures et métriques présentées correspondent aux fichiers de résultats fournis avec ce travail. Le corpus d'images n'est pas redistribué dans le dépôt décrit ici.

Références

[1] Y. Es Saady, A. Rachidi, M. El Yassa et D. Mammass, "AMHCD : A Database for Amazigh Handwritten Character Recognition Research," *International Journal of Computer Applications*, vol. 27, no. 4, pp. 44–48, août 2011, doi : 10.5120/3286-4475.

[2] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, nov. 1998, doi : 10.1109/5.726791.

[3] D. P. Kingma et J. Ba, "Adam : A Method for Stochastic Optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015. [En ligne]. Disponible : https://arxiv.org/abs/1412.6980.

[4] M. Abadi et al., "TensorFlow : A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.